Práctica 4

Nolan Tannion Rodríguez

Computación Avanzada
Universidad Autónoma de Madrid

Resumen

Simulación numérica del péndulo real forzado y amortiguado. Cálculo del exponente de Lyapunov variando las condiciones iniciales y el parámetro de rozamiento, obteniendo que el sistema cambia de régimen no caótico a caótico al variar el rozamiento. Se incluye una simulación del comportamiento de los péndulos, el espacio de fases y potencia espectral obtenida con la transformada de Fourier.

1. Introducción teórica

1.1. Péndulo real

Se considera el péndulo real amortiguado y forzado, cuya dinámica viene descrita por

$$\ddot{\theta} + q\dot{\theta} + \frac{g}{l}\sin\theta = A_F \sin(\Omega t), \tag{1}$$

donde θ es el desplazamiento angular, q el coeficiente de amortiguamiento, l la longitud del péndulo, g la aceleración de la gravedad, y $A_F \sin(\Omega t)$ la fuerza impulsora periódica de amplitud A_F y frecuencia Ω.

Definiendo la velocidad angular $\omega = \dot{\theta}$, el sistema se escribe como un sistema de ecuaciones de primer orden:

$$\begin{cases} \dot{\theta} = \omega \\ \dot{\omega} = -\dfrac{g}{l}\sin(\theta) - q\omega + A_F \sin(\Omega t). \end{cases} \tag{2}$$

Este sistema es no lineal y presenta una alta sensibilidad a las condiciones iniciales y parámetros.

1.2. Exponente de Lyapunov

La sensibilidad a condiciones iniciales se cuantifica mediante el exponente de Lyapunov λ.

$$|\delta\theta(t)| \approx |\delta\theta(0)|e^{\lambda t} \tag{3}$$

Siendo $\delta\theta(t)$ la separación entre dos trayectorias inicialmente próximas en el espacio de fases.

En función del signo de λ podemos estudiar la naturleza del sistema:

- $\lambda < 0$: las separaciones decrecen exponencialmente, por lo que el sistema es no caótico.
- $\lambda > 0$ las separaciones crecen con el tiempo. Esto indica que el sistema está en régimen caótico.
- $\lambda = 0$ el sistema se encuentra en la zona de transición entre regímenes.

Conociendo la diferencia angular en cada instante del tiempo, estimaremos λ haciendo un ajuste lineal a la gráfica:

$$\ln|\Delta\theta(t)| = \lambda\, t, \tag{4}$$

en la zona donde la separación crece aproximadamente de forma exponencial y antes de la saturación.

1.3. Integrador de Euler-Cromer

El método de Euler-Cromer es un esquema explícito de integración numérica adecuado para sistemas dinámicos de segundo orden.

Considerando el sistema

$$\dot{\theta} = \omega, \qquad \dot{\omega} = f(\theta, \omega, t), \tag{5}$$

el algoritmo actualiza primero la velocidad y luego la posición:

$$\omega_{n+1} = \omega_n + f(\theta_n, \omega_n, t_n)\Delta t,\ \theta_{n+1} \quad = \theta_n + \omega_{n+1}\Delta t. \tag{6}$$

A diferencia del método de Euler estándar, la posición se actualiza con la velocidad ya corregida, lo que mejora la estabilidad numérica en sistemas oscilatorios.

Para el péndulo amortiguado y forzado,

$$f(\theta, \omega, t) = -q\omega - \frac{g}{l}\sin\theta + F\sin(\Omega t). \tag{7}$$

La precisión global del método es de orden $\mathcal{O}(\Delta t)$ y su estabilidad depende del tamaño del paso temporal.

1.4. Transformada de Fourier

El análisis frecuencial de la señal angular $\theta(t)$ se basa en la transformada de Fourier continua,

$$\hat{\theta}(\omega) = \int_{-\infty}^{\infty} \theta(t) e^{-i\omega t} dt,$$

que descompone la señal en sus componentes armónicas.

En el caso numérico se dispone de una señal discreta muestreada uniformemente,

$$t_n = nh, \qquad n = 0, \ldots, N-1,$$

donde h es el paso temporal y $T = Nh$ la duración total observada. La Transformada Discreta de Fourier (DFT) viene dada por

$$\hat{\theta}(k) = \sum_{n=0}^{N-1} \theta_n e^{-i2\pi kn/N}, \qquad k = 0, \ldots, N-1. \tag{8}$$

Para señales reales se cumple simetría hermítica,

$$\hat{\theta}_{N-k} = \hat{\theta}_k^*$$

por lo que **rfft** devuelve únicamente las componentes de frecuencia positiva,

$$k = 0, \ldots, \frac{N}{2}$$

Las frecuencias angulares asociadas son

$$\omega_k = \frac{2\pi k}{Nh} \tag{9}$$

La resolución en frecuencia está determinada por la duración total de la señal:

$$\Delta f = \frac{1}{N\Delta t} \tag{10}$$

mientras que la frecuencia máxima accesible viene dada por la frecuencia de Nyquist:

$$f_{\max} = \frac{1}{2\Delta t} \tag{11}$$

El espectro de potencias se define como

$$P_k = |\hat{\theta}_k|^2 \tag{12}$$

y cuantifica la distribución de energía entre frecuencias. En dinámica periódica aparecen picos discretos en múltiplos de la frecuencia dominante; en régimen caótico se observa, además de posibles picos principales, un fondo continuo asociado a la redistribución no lineal de energía.

Para optimizar el rendimiento del algoritmo FFT se elige típicamente

$$N = 2^m,$$

lo que reduce el coste computacional a orden $\mathcal{O}(N \log N)$.

2. Implementación

2.1. Implementación general

Los distintos programas desarrollados comparten una estructura común: definición de parámetros físicos, definición del sistema dinámico, integración temporal mediante un método numérico y análisis o representación de los resultados.

2.1.1. Definición de variables y parámetros físicos

En primer lugar se definen las constantes del sistema y los parámetros asociados de la fuerza impulsora. Estos valores determinan el régimen dinámico del péndulo.

```
# Variables y constantes
g = 9.81
l = 9.81
omega_forz = 2/3
Af = 1.2
roz = 0.5
```

Estos parámetros se agrupan posteriormente en un vector que se pasa al integrador numérico.

```
parametros = [Af, omega_forz, roz]
```

2.1.2. Coeficientes de rozamiento

Para analizar la sensibilidad del sistema frente a variaciones del amortiguamiento se utilizan distintos valores del coeficiente de rozamiento q en los diferentes scripts.

En primer lugar, ya se ha definido el valor de referencia de 0.5.

Para el cálculo de la práctica base se ha utilizado:

```
rozamientos = np.linspace(0.3, 0.9, 4)
```

Mientras que en la transformada de Fourier y el espacio de fases se usa:

```
rozamientos = np.linspace(0.30, 0.90, 3)
```

En la simulación se aumenta el número de puntos del linspace a 6.

2.1.3. Definición del sistema dinámico

La dinámica del péndulo amortiguado y forzado se implementa mediante una función que devuelve las derivadas de las variables dinámicas. El estado del sistema está definido por el vector $(\theta, \dot{\theta})$.

```
def sistema(t, vars, params):
    dvars = np.zeros_like(vars)

    dvars[0] = vars[1]
    dvars[1] = -g/l*np.sin(vars[0]) - params
        [2]*vars[1] + params[0]*np.sin(params
        [1]*t)

    return dvars
```

Esta función representa el sistema de ecuaciones diferenciales que describe el movimiento del péndulo real.

2.1.4. Condiciones iniciales

Las condiciones iniciales del primer péndulo son:

```
theta01 = 0.20
omega01 = 0.00
estado1 = [theta01, omega01]
```

Se definen las condiciones del segundo péndulo con una pequeña diferencia angular:

```
theta02 = theta01 - 1e-2
omega02 = 0.0
estado2 = [theta02, omega02]
```

Excepto para la simulación, que se utiliza:

```
theta02 = theta01 + 1e-4
```

2.1.5. Integrador Euler-Cromer

La función implementada recibe la función que define el sistema dinámico, el intervalo temporal, el estado inicial y los parámetros físicos del sistema.

```
def euler_cromerr(sistema, t0, tf, estado0,
    params, h = 1e-3):

    N = int(abs(tf - t0) / h)
    vars = np.zeros((N, 2))
    vars[0] = estado0
    t = [t0]

    for i in range(N - 1):
        deriv = sistema(t[i],vars[i], params)

        vars[i+1] = vars[i]

        # velocidades
        vars[i+1, 1::2] = vars[i, 1::2] + h *
            deriv[1::2]

        # posiciones (usan la velocidad
            actualizada)
        vars[i+1, 0::2] = vars[i, 0::2] + h *
            vars[i+1, 1::2]

        t.append(t[i] + h)

    return t, vars
```

El estado del sistema se almacena en un array que contiene la evolución temporal de las variables dinámicas $(\theta, \dot{\theta})$, mientras que el vector t recoge los instantes de tiempo correspondientes. Se llama a la función con la estructura:

```
t, sol = euler_cromerr(sistema=sistema, t0=t0,
     tf=tmax, estado0=estado0, params=params,
    h=h)
```

```
theta = sol[:,0]
omega = sol[:,1]
```

Se lleva a cabo una normalización $\theta \in [-\pi, \pi]$ con el operador resto para el cálculo del exponente:

```
thetanorm = (theta + np.pi) % (2*np.pi) - np.
    pi
```

2.2. Exponente de Lyapunov

Tras calcular la evolución temporal de ambos sistemas, se obtiene la diferencia angular entre las trayectorias:

```
diff = np.abs(theta2norm - theta1norm)
diff = np.log(diff)
```

El exponente de Lyapunov se estima realizando un ajuste lineal de $\log(\Delta\theta)$ en función del tiempo durante la fase de crecimiento lineal. Se considera el tramo del 20 % al 40 % del array para el ajuste.

```
N =int( 0.2* len(t1))
tly = np.float32(t1[N:2*N])
diffly = diff[N:2*N]

p = np.polyfit(tly, diffly, 1)
```

El coeficiente de la pendiente almacenado en $p[0]$ corresponde a una estimación de λ.

2.2.1. Dependencia del rozamiento

Al calcular el coeficiente se distinguen dos casos:

- Distintas condiciones iniciales y rozamientos
- Distintas condiciones iniciales, mismo rozamiento

En el primer caso el péndulo de referencia tiene un valor fijo del rozamiento y el segundo péndulo unas condiciones iniciales y rozamiento diferentes. En el segundo caso el péndulo de referencia y el segundo tienen el mismo rozamiento pero distintas condiciones. Ambos casos se estudian para distintos rozamientos.

En el primer caso se implementa dejando el primer péndulo fuera del bucle de rozamientos y calculandolo una sola vez:

```
t1, sol1 = euler_cromerr(sistema=sistema, t0 =
     t0, tf = tmax, estado0= estado1, params=
    parametros0, h = dt)
for rozm in rozamientos:
    parametros[2] = rozm
    t2, sol2 = euler_cromerr(sistema=sistema,
        t0 = t0, tf = tmax, estado0= estado2,
        params= parametros, h = dt)
```

```
for rozm in rozamientos:
    parametros[2] = rozm
    t1, sol1 = euler_cromerr(sistema=sistema,
        t0 = t0, tf = tmax, estado0= estado1,
        params= parametros, h = dt)
    t2, sol2 = euler_cromerr(sistema=sistema,
        t0 = t0, tf = tmax, estado0= estado2,
        params= parametros, h = dt)
```

2.3. Simulación

Para la simulación que evolucione con el tiempo usamos la función:

```
from matplotlib.animation import FuncAnimation
```

Siguiendo la documentación [1], debemos definir unos objetos para cada elemento a representar, una función para el fotograma inicial y una función de actualización que será llamada en cada frame.

```
# Representamos los pendulos como segmentos
    con borde redondeado
lines = [ax.plot([], [], 'o-', label =
    leyendas[_])[0] for _ in range(len(
    rozamientos))]
lines.append(ax.plot([], [], 'o-', color= '
    black', label = leyendas[-1])[0])  #
    Primer pendulo graficado de color negro

# Situamos la leyenda fuera del cuadro de la
    grafica para mejor visibilidad
ax.legend(fontsize=7, loc='upper left',
    bbox_to_anchor=(1.05, 1), borderaxespad=0)
# Valor de la fuerza en funcion del tiempo
Valorf = ax.text(0.05, 0.95, '', transform=ax.
    transAxes, color='red', fontsize=9, va='
    top')
```

Hemos creado una linea para cada rozamiento con un for que recorre los rozamientos y añadimos con append el péndulo de referencia. Hemos creado también un texto que indicará el valor de la fuerza.

```
transform = ax.transAxes
```

Esta sección indica que los valores de la ubicación del texto son relativos al eje y no valores numéricos.

Finalmente las funciones:

```

# Grafica inicial
def init():
    for line in lines:
        line.set_data([], [])
    return lines


# Funcion para actualizar la animacion
def animate(frame):
    for i, line in enumerate(lines):
        line.set_data([0, x[i, frame]], [0, y[
            i, frame]])

    # Valor de la fuerza en este instante
    F_t     = Af * np.sin(omega_forz * t1[frame
        ])
    # Ajustamos el texto de la fuerza y el
        titulo
    Valorf.set_text(f'F(t) = {F_t:.2f}')
    ax.set_title(f'Evolucion de los pendulos.
        t = {t1[frame]:.2f}')

    return lines + [Valorf]

# Llamamos a la funcion para la animacion con
    los parametros correspondientes
ani = FuncAnimation(
    fig,
    animate,
    frames=len(t1),
    init_func=init,
    interval=10,
    blit=False
)

# Almacenamos la simulacion (mucho mas lento
    que visualizarla solo)
#ani.save('Pendulos_sim.mp4', writer='ffmpeg')
plt.show()
```

2.4. Espacio de fases

Para el espacio de fases simplemente tomamos los datos de el ángulo y velocidad angular obtenidos en la integración y los representamos en una gráfica. Usamos scatter y no plot aprovechando la alta resolución de puntos para incluir un gradiente de color que represente el instante en el tiempo al que corresponde el punto en el espacio de fases. Usamos un panel de figuras para mayor claridad.

```
    fig, ax = plt.subplots(nrows=2, ncols=2,
        figsize = (10,6))
figura, eje = plt.subplots(figsize = (7,7))
fig.subplots_adjust(wspace=0.4, hspace=0.4)
j = 0
k = 0
for i in range(len(thetasol)):
    theta = thetasol[i]
    omega = omegasol[i]

    if j>1:
```

```
        j = 0
        k = 1

    eje.scatter(theta,omega, s = 0.4,  label = f'{leyendas[i]}')

    # Representamos el espacio de fases usando un gradiente de color para
    # indicar la evolucion temporal
    e_fases = ax[j,k].scatter(theta, omega, s=0.4, label = f'{leyendas[i]}', c = t1, cmap = 'viridis')
    ax[j,k].legend()
    ax[j,k].set_xlabel(r"$\theta$")
    ax[j,k].set_ylabel(r"$\dot{\theta}$")
    fig.colorbar(e_fases, ax = ax[j,k], label = 't') # Barra que muestra el valor del gradiente
    j += 1
```

2.5. Cálculo de la transformada de Fourier

Para el desarrollo del código, la información expuesta en clases se ha complementado con la bibliografía [2] de la librería numpy.

2.5.1. Elección del intervalo temporal

El paso temporal se define en función del periodo de la fuerza impulsora,

```
T_forz = 2*np.pi/omega_forz
h = T_forz/100
```

De este modo se utilizan aproximadamente 100 puntos por periodo, lo que permite resolver adecuadamente la dinámica temporal y evitar falta de resolución.

El tiempo total de integración se fija como un múltiplo del periodo:

```
T_total = 2000*T_forz
```

2.5.2. Eliminación del régimen transitorio

Antes de calcular la transformada se descarta una fracción inicial de la señal:

```
frac_trans = 0.3
i0 = int(frac_trans * len(thetasol[0]))
theta_est = thetasol[0][i0:]
```

De esta forma el análisis se realiza únicamente sobre el régimen estacionario.

2.5.3. Selección del número de puntos para la FFT

Para optimizar el cálculo de la FFT se selecciona el mayor número de puntos que sea potencia de dos y menor que la longitud del array estacionario:

```
N_disp = len(theta_est)
m = int(np.floor(np.log2(N_disp)))
N_fft = 2**m
```

Esta elección mejora la eficiencia computacional del algoritmo FFT.

2.5.4. Cálculo del espectro

La transformada se calcula sobre los últimos N_{FFT} puntos de la señal:

```
theta_fft = theta[-N_fft:]
espectros.append(np.abs(rfft(theta_fft))**2)
```

Se toma el módulo al cuadrado para obtener la potencia espectral.

Las frecuencias asociadas se obtienen mediante la función implementada en numpy.

```
freq = rfftfreq(N_fft, d=h) * 2*np.pi
```

2.5.5. Organización de las subfiguras

Las gráficas individuales se organizan en un panel 2×2. Para transformar el índice lineal del bucle en coordenadas del panel se utilizan las operaciones módulo y división entera:

```
j, k = idx % 2, idx // 2
```

El operador módulo determina la fila y la división entera determina la columna del subplot correspondiente.

3. Resultados

3.1. Coeficientes de Lyapunov

3.1.1. Distintas condiciones y varios rozamientos

Los coeficientes obtenidos según el rozamiento en los ajustes de la Figura 7 son:

- $roz = 0{,}40$, $\lambda = 0{,}1786$
- $roz = 0{,}50$, $\lambda = 0{,}1967$
- $roz = 0{,}60$, $\lambda = 0{,}1857$
- $roz = 0{,}70$, $\lambda = 0{,}1711$

Todos son positivos, incluyendo el caso de mismo rozamiento, lo que significa que las trayectorias de las partículas divergen. Esto muestra la sensibilidad a las condiciones iniciales.

En la Figura 1 vemos la evolución de los distintos péndulos. Se indica el péndulo de referencia y los distintos rozamientos del segundo péndulo con condiciones iniciales distintas. Para rozamientos distintos al de referencia, el periodo transitorio es prácticamente inexistente y comienza la divergencia.

3.1.2. Mismas condiciones y rozamiento

Variando las condiciones pero manteniendo el mismo rozamiento en ambos péndulos obtenemos los ajustes de la Figura 2:

- $roz = 0{,}300$, caótico, $\lambda = 0{,}2138$
- $roz = 0{,}500$, caótico, $\lambda = 0{,}1967$
- $roz = 0{,}700$, no caótico, $\lambda = -0{,}2842$
- $roz = 0{,}900$, no caótico, $\lambda = -0{,}5232$

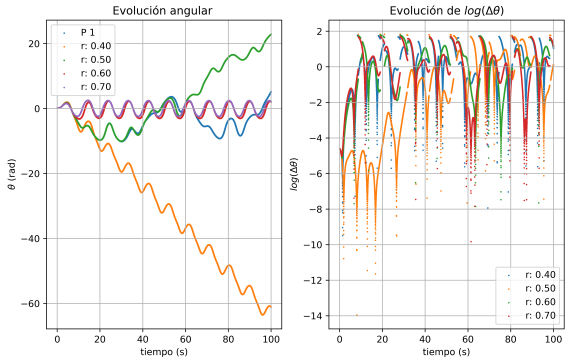

Figura 1: En cada panel se representa el logaritmo de la diferencia angular frente al tramo de tiempo elegido para el ajuste según cada rozamiento.

Vemos que el coeficiente disminuye al aumentar el rozamiento, llegando a cambiar de signo por lo que hay una transición de caótico a no caótico. Esto es coherente ya que al aumentar la disipación las trayectorias tienden a diverger menos.

El caso estudiado en el apartado anterior con rozamiento $1 = 0{,}5$ nos devuelve el mismo resultado como era esperado.

En la Figura 5 se incluye la evolución del logaritmo de la diferencia. Se ha calculado que al aumentar el rozamiento, la duración del periodo transitorio disminuye. De nuevo esto es un resultado de la misma naturaleza que el anterior, al aumentar la disipación las trayectorias de transición se acortan y domina la dinámica estacionaria del sistema.

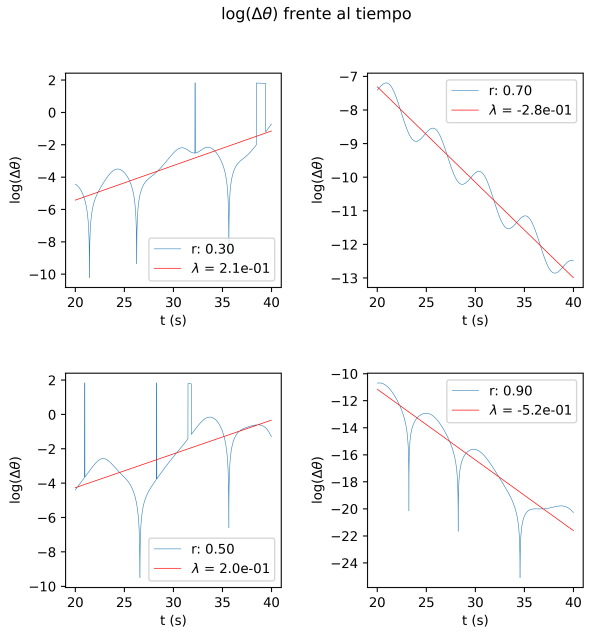

Figura 2: Logaritmo de la diferencia angular frente al tiempo con su correspondiente ajuste lineal que representa el coeficiente de Lyapunov.

3.2. Simulación

Se puede visualizar la simulación ejecutando el código o visualizarla en `https://youtu.be/wHX8HSEfj68` ya compilada y almacenada.

Se puede observa el régimen caótico del sistema viendo la evolución gráfica.

3.3. Espacio de fases

Los espacios de fases de cada péndulo en la Figura 3. Los péndulos con exponentes de Lyapunov positivos muestran trayectorias no cerradas y erráticas propias de sistemas caóticos. Por otra parte los sistemas donde el exponente ha sido negativo muestran una trayectoria con una estructura mucho más simple.

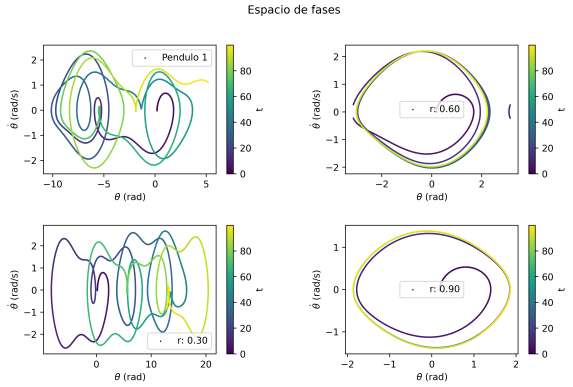

Figura 3: Espacio de fases para cada set de condiciones iniciales. El péndulo superior izquierda es el péndulo de referencia. Superior derecha mismo rozamiento con distintas condiciones iniciales. En los inferiores varía el rozamiento, siendo menor en la izquierda y mayor en la derecha.

3.4. Transformada de Fourier

Vemos en la Figura 4 la potencia espectral de cada uno de los péndulos. Para la frecuencia principal el pico principal está en la frecuencia impulsora, además un pico secundario en el 3º armónico. Entre el 0 y el 2º armónico se aprecia un continuo de potencia distribuida en otras frecuencias no principales. Esto nos indica que el sistema se encuentra en un régimen cercano a la transición.

Con menor rozamiento el pico principal está desplazado del segundo armónico. Aparecen también activaciones del primer y tercer armónico a la vez que picos en frecuencias intermedia. Este comportamiento también es característico de sistemas caóticos.

Para los rozamientos mayores, tenemos un pico principal en la frecuencia impulsora. En el rozamiento $r = 0{,}6$ hay una muy baja activación del segundo armónico, que desaparece al seguir aumentando el rozamiento. No se aprecian otras activaciones además de estas, lo que es propio de sistemas no caóticos.

En la Figura 6 se representa la potencia espectral de todos los sistemas en una sola gráfica.

4. Conclusiones

Los resultados obtenidos son consistentes con las predicciones de nuestro marco teórico.

Los sistemas con distintas condiciones y rozamientos divergen rápidamente. Los sistemas con distintas condiciones pero mismos rozamientos cambian de régimen caótico

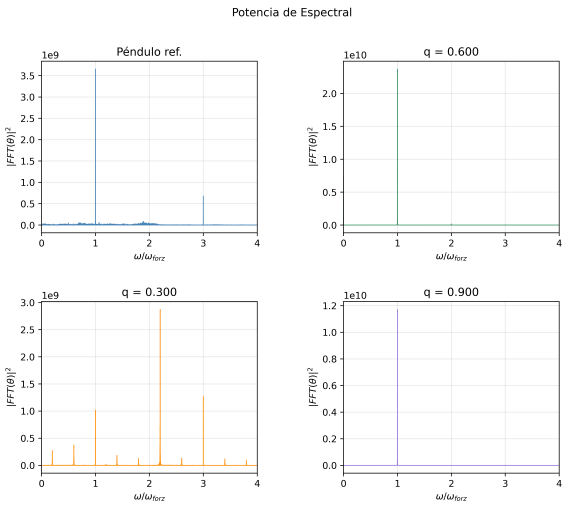

Figura 4: Potencia espectral en función de la frecuencia entre la frecuencia impulsora para cada set de condiciones.

a no caótico al aumentar el rozamiento, fenómeno que se puede apreciar además en la potencia espectral.

Referencias

[1] Matplotlib Development Team. matplotlib.animation.funcanimation. https://matplotlib.org/stable/api/_as_gen/matplotlib.animation.FuncAnimation.html, 2024. Accessed: 2026-03-04.

[2] NumPy Developers. Discrete fourier transform (numpy.fft). https://numpy.org/devdocs/reference/routines.fft.html, 2025. NumPy Reference Documentation. Accessed: 2026-03-04.

Gráficas

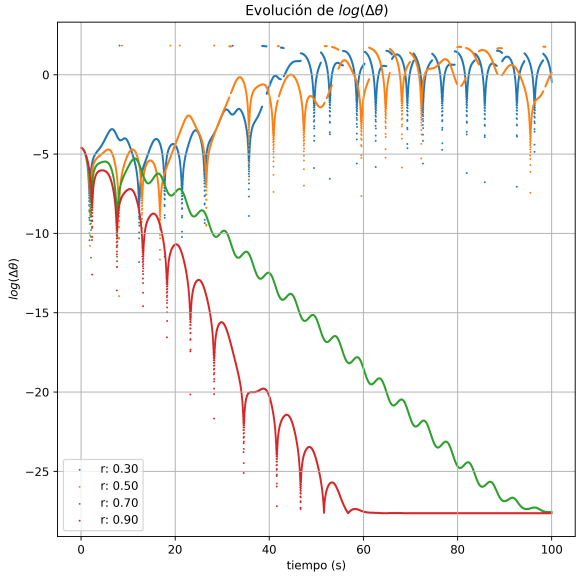

Figura 5: La figura muestra el logaritmo de la diferencia angular en función del tiempo.

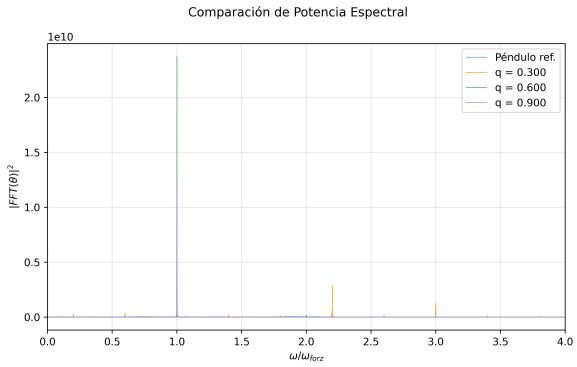

Figura 6: Potencias espectrales de todos los péndulos representadas en una sola gráfica.

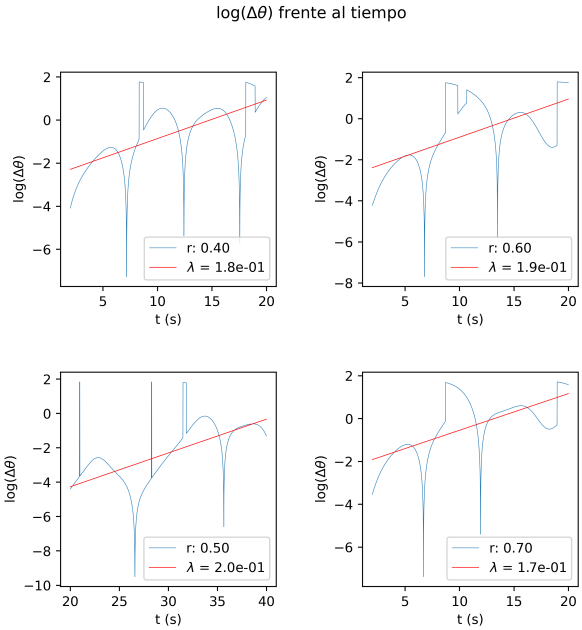

Figura 7: La figura de la izquierda representa la evolución angular de cada péndulo. En la derecha se representa el logaritmo de la diferencia frente al tiempo.